Manual Técnico
Módulo Vidrio

Versión 1.0 | 22/05/2026

Documentación técnica para desarrolladores y administradores del sistema.

Índice de Contenido

#	Sección
1.	Arquitectura del Sistema
2.	Estructura de Archivos
3.	Base de Datos
3.1	Esquema de la Tabla
3.2	Relaciones
4.	Dependencias
5.	Módulos y Clases Principales
5.1	DBManager
5.2	PantallaVidrio
5.3	Importador Excel
6.	Flujo de Datos
6.1	Guardado Manual
6.2	Edición
6.3	Eliminación Directa
6.4	Importación Excel
7.	Manejo de Errores
8.	Logging y Depuración
9.	Configuración

1. Arquitectura del Sistema

El Módulo de Vidrio sigue una arquitectura de tres capas:

Capa	Tecnología	Responsabilidad
Presentación	PyQt6	Interfaz gráfica (PantallaVidrio)
Lógica de Negocio	Python 3.12	DBManager, validaciones
Acceso a Datos	pyodbc + SQL Server	CRUD sobre materiales_vidrio
Almacenamiento	SQL Server	Base de datos 'vidrio', tabla materiales_vidrio

[Imagen pendiente: arquitectura_sistema.png]

2. Estructura de Archivos

Archivo	Descripción
pantalla_vidrio.py	UI principal, modos de operación, eventos
db_manager.py	CRUD, conexión BD, context manager
cargar_excel_vidrio.py	Importador Excel con MERGE set-based
imprimir.py	Generación de PDF con reportlab
utils.py	Función dec2() para conversión a Decimal

[Imagen pendiente: estructura_modulos.png]

3. Base de Datos

3.1 Esquema de la Tabla

Columna	Tipo	Nulo	Descripción
Clave	varchar(20)	NO	PK - Clave única del material
Descripcion	varchar(100)	YES	Nombre del material
Precio	decimal(10,2)	YES	Precio unitario por m²

[Imagen pendiente: diagrama_bd.png]

3.2 Relaciones

- La tabla NO tiene claves foráneas explícitas.
- La clave primaria es 'Clave' (única).
- No hay relaciones directas con otras tablas.
- La integridad se maneja a nivel de aplicación.

4. Dependencias

Librería	Versión	Uso
PyQt6	6.5+	Interfaz gráfica completa
pyodbc	4.0+	Conexión ODBC a SQL Server
openpyxl	3.0+	Lectura de archivos Excel
pandas	1.5+	Procesamiento de datos en importador
numpy	1.24+	Manejo de NaN en importador
reportlab	3.6+	Generación de PDF para impresión
win32print	pywin32	Impresión directa a impresora física
Python	3.10+	Lenguaje base

```
pip install PyQt6 pyodbc openpyxl pandas numpy reportlab pywin32
```

5. Módulos y Clases Principales

5.1 DBManager (db_manager.py)

- listar_todos() → List[Dict] - Obtiene todos los materiales
- existe_clave(clave) → bool - Verifica existencia
- insertar(reg) - Inserta nuevo material
- actualizar_por_clave(clave_original, reg) - Actualiza (permite cambiar clave)
- eliminar_por_clave(clave) - Elimina material
- _get_conn() - Context manager para conexiones seguras

```
# Ejemplo de uso
db = DBManager()
material = {
    "clave": "V-CLARO-6",
    "descripcion": "Vidrio Claro 6mm",
    "precio": Decimal("450.00")
}
db.insertar(material)
```

5.2 PantallaVidrio (pantalla_vidrio.py)

- _on_guardar() - Guarda material (modo nuevo o edición)
- _modo_editar() - Activa modo edición, habilita búsqueda
- _on_eliminar_directo() - Elimina el registro cargado en inputs
- Inicio() - Limpia todos los campos y tabla
- _buscar_dinamico() - Filtra tabla por prefijo de clave
- _on_imprimir() - Diálogo de impresión (Pantalla/PDF/Impresora)
- _llenar_tabla() - Carga datos en QTableWidget
- _cargar_desde_seleccion() - Carga automática al seleccionar fila
- eventFilter() - Valida clave duplicada al salir del campo
- _db_get_por_clave() - Fallback para obtener registro por clave

```
# Atajos de teclado definidos
Ctrl+S → _on_guardar
Ctrl+F → focus búsqueda
Ctrl+P → _on_imprimir
Delete → _on_eliminar_directo
F5 → _on_refresh
Esc → close
```

5.3 Importador Excel (cargar_excel_vidrio.py)

- importar(file_path, sheet_name) - Función principal
- _find_header_row() - Detecta fila de encabezados
- _pick_columns() - Identifica columnas Clave, Descripción, Precio
- _guess_text_column() - Infiere columna de descripción si no existe
- _prepare_rows() - Prepara datos y valida
- _bulk_merge() - Ejecuta MERGE con tabla temporal #imp
- _save_rejected() - Guarda registros rechazados en CSV

```
# Uso del importador
python -m materiales.vidrio.cargar_excel_vidrio
# o con parámetros:
python -m materiales.vidrio.cargar_excel_vidrio "ruta\archivo.xlsx" "NombreHoja"
```

6. Flujo de Datos

6.1 Guardado Manual

- 1. Usuario completa campos (Clave, Descripción, Precio)
- 2. Usuario presiona Guardar (Ctrl+S)
- 3. UI valida campos obligatorios (Clave, Descripción, precio > 0)
- 4. Si modo='nuevo': verifica que clave no exista
- 5. Si modo='editar': verifica duplicado si cambia clave
- 6. UI llama a DBManager.insertar() o actualizar_por_clave()
- 7. DBManager ejecuta INSERT/UPDATE con pyodbc
- 8. UI muestra mensaje de éxito y limpia pantalla

[Imagen pendiente: flujo_datos.png]

6.2 Edición (con cambio de clave permitido)

- 1. Usuario presiona Editar
- 2. UI activa barra de búsqueda
- 3. Usuario escribe prefijo de clave
- 4. UI filtra y muestra resultados
- 5. Usuario hace clic o doble clic en una fila
- 6. UI carga datos en campos (todos habilitados)
- 7. Usuario modifica clave, descripción o precio
- 8. Usuario presiona Guardar
- 9. Si clave cambió, verifica que la nueva no exista
- 10. DBManager ejecuta UPDATE
- 11. UI muestra mensaje de éxito

6.3 Eliminación Directa

- 1. Usuario carga el material a eliminar (buscando o escribiendo clave)
- 2. Usuario presiona Eliminar (Delete)
- 3. UI verifica que la clave exista en BD
- 4. UI muestra diálogo de confirmación

- 5. Usuario confirma
- 6. UI llama a DBManager.eliminar_por_clave()
- 7. UI muestra mensaje 'Registro eliminado.'
- 8. UI limpia pantalla

6.4 Importación Excel (MERGE set-based)

- 1. Usuario ejecuta script: python -m materiales.vidrio.cargar_excel_vidrio
- 2. Script lee Excel y detecta encabezados
- 3. Identifica columnas Clave, Descripción, Precio
- 4. Valida cada fila (clave no vacía, precio válido)
- 5. Crea tabla temporal #imp
- 6. Ejecuta MERGE contra materiales_vidrio:
- - MATCHED → UPDATE (precio = MAX(existente, nuevo))
- - NOT MATCHED → INSERT
- 7. Registros rechazados se guardan en CSV
- 8. Muestra resumen en consola

7. Manejo de Errores

Nivel	Mecanismo	Ejemplo
Validación UI	Comprobaciones en _on_guardar()	Clave vacía → mensaje
Clave duplicada	existe_clave() + QMessageBox	Mensaje de advertencia
Conexión BD	try/except + context manager	Logging + raise
Eliminación	verificación existencia + confirmación	Mensaje si no existe
Importación	try/except por registro + CSV rechazados	Continúa con siguiente
Impresión	try/except en imprimir_pdf_guardar	QMessageBox.critical

```
# Ejemplo de context manager en db_manager.py
@contextmanager
def _get_conn(self):
    conn = pyodbc.connect(self.conn_str)
    try:
        yield conn
    finally:
        conn.close()
```

8. Logging y Depuración

- No hay archivo de log específico para este módulo
- Los errores de conexión se capturan y muestran en UI
- Los rechazados de importación se guardan en:
- D:\vah\logs\rechazados_vidrio_YYYYMMDD_HHMMSS.csv
- Para depuración, ejecutar el módulo desde consola:
- python -m materiales.vidrio.pantalla_vidrio

```
# Ejemplo de salida de importador
Columnas detectadas → Clave: 'CLAVE' | Descripción: 'DESCRIPCIÓN' | Precio: 'PRECIO'
=== Importación VIDRIO ===
Válidos: 25  |  Insertados: 10  |  Actualizados: 15
Rechazados: 2
Detalle rechazados: D:\vah\logs\rechazados_vidrio_20260522_143022.csv
```

9. Configuración

Cadena de conexión a SQL Server (db_manager.py):

```
cs = (
    f"DRIVER={{ODBC Driver 18 for SQL Server}};"
    f"SERVER={self.server};"
    f"DATABASE={self.database};"
    "Trusted_Connection=yes;"
    "Encrypt=yes;TrustServerCertificate=yes;"
)
```

Configuración por defecto:

```
DEFAULT_SERVER = r"DESKTOP-AE60PIU\SQLEXPRESS"
DEFAULT_DATABASE = "vidrio"
DEFAULT_TABLE = "dbo.materiales_vidrio"
DEFAULT_XLSX = r"C:\Users\Personal\OneDrive\Escritorio\vidrio.xlsx"
```

— Fin del Manual Técnico —

Documento generado el 22/05/2026 | Versión 1.0

VAH ERP © Todos los derechos reservados